

MODULE 3

Advanced Design Patterns &
Technology Integration

Module 3: Advanced Design Patterns & Technology Integration

Module Overview

Module Title: Advanced Design Patterns & Technology Integration

Objective: Dive deep into specific modern technologies and design patterns that will be used in the implementation phase of the Smart Inventory and Logistics Platform.

Key Learning Outcomes:

- Understand and apply Polyglot Persistence patterns to match database technologies to specific data requirements
- Design and implement three distinct AI integration patterns for different use cases
- Architect integration solutions using API-led connectivity, event-driven architecture, and GraphQL
- Address challenges of data synchronization and transactional consistency across heterogeneous systems

Connection to Course Project: Students will apply these advanced patterns to enhance their TO-BE designs, determining which databases are appropriate for different data types, where AI can add value, and how components will integrate in their final architecture.

Introduction to Module 3

Welcome to Module 3. In Modules 1 and 2, we established the foundations of enterprise architecture and designed the four architectural views for our Smart Inventory and Logistics Platform. We defined business capabilities, modeled data entities, created C4 application diagrams, and selected technology stacks.

Now, in Module 3, we move beyond foundational concepts to explore advanced patterns that will make our system truly modern and capable. This module is where theory meets cutting-edge practice. We'll dive deep into three critical areas that define

contemporary system design: Polyglot Persistence, AI Integration, and Integration Architecture.

These aren't just academic concepts—they're practical responses to real-world challenges. Polyglot persistence recognizes that no single database is optimal for all use cases. AI integration acknowledges that modern systems must learn and predict, not just transact. Integration architecture addresses the reality that no system exists in isolation—your platform must connect with HRIS, procurement, finance, and countless other systems.

By the end of this module, you'll understand not just what these patterns are, but when to apply them, how to implement them, and what trade-offs they involve. You'll be equipped to make sophisticated architectural decisions that balance capability, complexity, and cost.

Let's begin our exploration of these advanced patterns.

Section 3.1: Polyglot Persistence

The Concept: Why Different Databases for Different Jobs?

Polyglot persistence is the practice of using multiple database technologies within a single system, each chosen to best serve specific data requirements. The term draws an analogy with polyglot programming—using multiple programming languages because no single language is optimal for all tasks.

The fundamental insight is simple: different data has different characteristics, access patterns, consistency requirements, and lifecycle needs. A relational database optimized for ACID transactions is terrible for storing hierarchical product catalogs with variable attributes. A document store perfect for product catalogs is terrible for complex financial reporting requiring multi-table joins. A time-series database optimized for IoT sensor data is terrible for tracking relationships between entities.

Yet traditional architecture often forces all data into a single database technology—typically relational—because that's what architects know or because "one size fits all" seems simpler. The result is compromise: the database does many things adequately but nothing exceptionally well.

Polyglot persistence rejects this compromise. Instead, it embraces heterogeneity, matching database to data requirement just as you'd match tool to task.

When to Consider Polyglot Persistence

Polyglot persistence becomes valuable when your system exhibits:

Data Diversity: You manage fundamentally different types of data—transactional records with rigid structure, content with flexible schema, time-series measurements, relationship graphs, full-text search indexes.

Varying Consistency Requirements: Some data demands immediate consistency (inventory counts during checkout), while others can tolerate eventual consistency (product catalog updates).

Different Access Patterns: Some data is accessed through complex queries and joins; some through key-value lookups; some through full-text search; some through time-range aggregations.

Scale Heterogeneity: Different data types scale differently. Transaction volume might grow linearly with business, while IoT data might explode exponentially.

Specialized Capabilities: Some databases offer unique capabilities—geospatial indexing, graph traversal, time-series compression—that are difficult to replicate in general-purpose databases.

Database Types and Their Use Cases

Relational Databases (SQL)

Characteristics:

- ACID compliance ensuring transaction integrity
- Strong consistency guarantees
- Complex join capabilities
- Schema enforcement
- Mature tooling and broad expertise

Optimal Use Cases:

- Financial transactions (orders, payments)
- Inventory counts requiring immediate consistency

- User accounts and authentication
- Audit logs requiring strict integrity
- Any data where referential integrity is critical

For the Smart Inventory Platform:

Use PostgreSQL for orders, financial transactions, and core inventory state. When a customer places an order, you need absolute certainty that inventory was decremented correctly and payment was processed—no eventual consistency here.

Document Stores (MongoDB)

Characteristics:

- Flexible schema (schema-on-read)
- Nested document structures
- Good performance for hierarchical data
- Horizontal scaling
- JSON/BSON data format

Optimal Use Cases:

- Product catalogs with varying attributes
- Content management
- User profiles with optional fields
- Configuration data
- Any data where structure varies across instances

For the Smart Inventory Platform:

Use MongoDB for the product catalog. Different products have different attributes—clothing has size and color, electronics has specifications, furniture has dimensions. A document model handles this naturally while a relational model would require complex entity-attribute-value tables or constant schema migrations.

Time-Series Databases (InfluxDB, TimescaleDB)

Characteristics:

- Optimized for timestamped data
- High write throughput
- Efficient time-based aggregations
- Data retention policies
- Downsampling and compression

Optimal Use Cases:

- IoT sensor readings
- Application performance metrics
- Network monitoring data
- Financial tick data
- Any data where time is the primary dimension

For the Smart Inventory Platform:

Use TimescaleDB (PostgreSQL extension) or InfluxDB for IoT sensor data from warehouses—temperature readings, humidity levels, motion detection events. These generate massive volume but are rarely queried individually; instead, you query averages, trends, and anomalies over time windows.

Search Engines (Elasticsearch)

Characteristics:

- Full-text search capabilities
- Inverted indexes for rapid searching
- Faceted search and filtering
- Fuzzy matching and relevance scoring
- Analytics aggregations

Optimal Use Cases:

- Product search
- Log analysis
- Documentation search
- Any application requiring "Google-like" search

For the Smart Inventory Platform:

Use Elasticsearch for fast product and inventory lookup. Employees need to find assets by partial names, descriptions, or even misspellings. A relational database with LIKE queries won't scale or provide relevance ranking.

Graph Databases (Neo4j, Amazon Neptune)

Characteristics:

- Optimized for highly connected data
- Traversal performance independent of graph size
- Relationship as first-class citizen

- Pattern matching queries

Optimal Use Cases:

- Supply chain relationships
- Organizational hierarchies
- Recommendation engines
- Fraud detection
- Any domain where relationships are complex

For the Smart Inventory Platform:

Consider Neo4j for supply chain relationships—suppliers providing parts, parts used in products, products stored in warehouses, warehouses serving stores. Graph traversals can answer questions like "which suppliers affect which stores if a part is delayed" efficiently.

Key-Value Stores (Redis, DynamoDB)

Characteristics:

- Extremely low latency
- Simple data model (key → value)
- High throughput
- Often in-memory

Optimal Use Cases:

- Caching
- Session storage
- Real-time counters
- Shopping carts
- Any data requiring fastest possible access

For the Smart Inventory Platform:

Use Redis for caching frequently accessed data—product details, inventory counts for popular items, user sessions. Also use it for rate limiting and temporary locks during distributed transactions.

Polyglot Persistence Challenges

Polyglot persistence isn't free. It introduces significant challenges that architects must address.

Data Synchronization

When data lives in multiple databases, keeping it consistent is non-trivial. If product details change in MongoDB, Elasticsearch must be updated to reflect the change. If an order is placed in PostgreSQL, the inventory count in Redis must be updated.

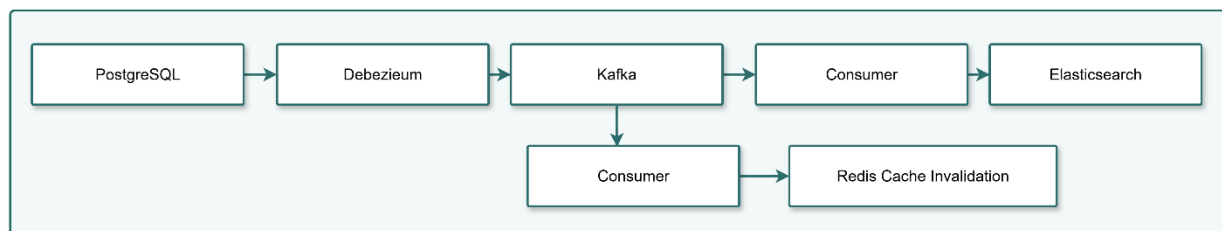
Synchronization Strategies:

Application-Level Synchronization:

The application writes to all relevant databases within a single transaction or operation. Simple but risky—if one write fails, data becomes inconsistent. Distributed transactions across heterogeneous databases are notoriously difficult.

Change Data Capture (CDC):

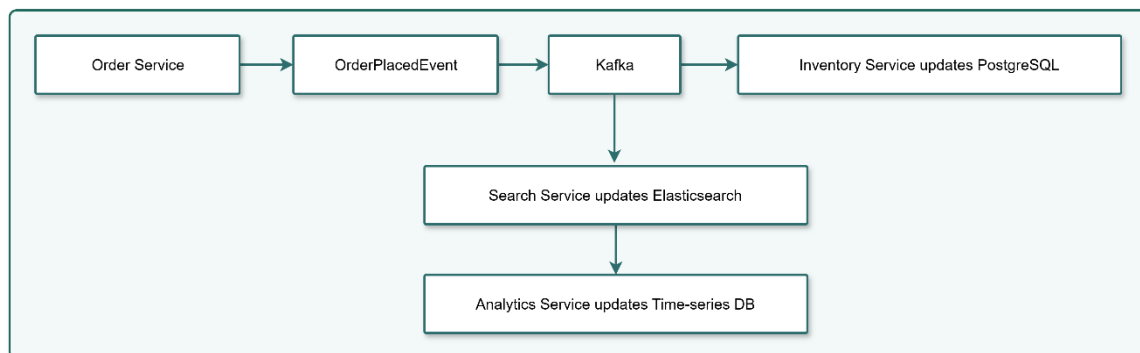
Capture changes in the primary database and propagate them asynchronously to other stores. Tools like Debezium read database transaction logs and publish change events to Kafka. Downstream consumers update secondary stores.



CDC provides loose coupling and resilience but introduces eventual consistency—there's a window where data is inconsistent.

Event-Driven Synchronization:

Applications publish domain events when significant state changes occur. Event subscribers update relevant databases. This is conceptually similar to CDC but events are emitted by the application, not captured from logs.



Transactional Consistency Across Databases

When a single business transaction spans multiple databases, maintaining atomicity is challenging. Consider order placement: you must decrement inventory in PostgreSQL, record the order, and update the search index. If the search update fails, should the order roll back?

Strategies for Cross-Database Transactions:

Distributed Transactions (XA):

Theoretically possible but practically problematic across heterogeneous databases. Performance suffers, and not all databases support XA.

Saga Pattern:

Break the transaction into a series of local transactions with compensating actions. If any step fails, execute compensating transactions to undo previous steps.

Order Saga:

1. Reserve inventory in PostgreSQL (local transaction)
2. If success, create order record in PostgreSQL
3. If success, update search index in Elasticsearch
4. If search update fails, execute compensating transaction to release inventory

The saga pattern ensures eventual consistency without distributed locks but adds complexity.

Eventual Consistency Acceptance:

For many use cases, eventual consistency is acceptable. If the search index is a few seconds behind, users might not notice. If inventory counts in Redis are slightly stale, occasional over-selling can be caught at order validation. Accepting eventual consistency simplifies architecture dramatically.

Query Federation

When data is distributed across databases, queries that need to combine data from multiple sources become complex. A reporting query might need order data from PostgreSQL, product attributes from MongoDB, and sensor readings from TimescaleDB.

Strategies for Federated Queries:

Application-Level Joins:

Query each database separately and combine results in application code. Simple but inefficient for complex queries.

Command Query Responsibility Segregation (CQRS):

Maintain separate data models for reads and writes. Create denormalized read-optimized views that combine data from multiple sources. Update these views asynchronously via events.

GraphQL Federation:

Use GraphQL as a unified query layer that fetches from multiple underlying databases. GraphQL resolvers know which database to query for each field.

Data Lake/ Warehouse:

For analytical queries, periodically extract data from all operational databases into a centralized warehouse (Redshift, BigQuery, Snowflake) where it can be joined freely.

Operational Complexity

Managing multiple databases increases operational burden. You need expertise in each database, monitoring for each, backup/recovery procedures for each, and deployment pipelines for each. This complexity must be justified by corresponding benefits.

Polyglot Persistence in the Smart Inventory Platform

For our platform, a polyglot approach makes sense:

Data Type	Database	Justification
Orders, Transactions	PostgreSQL	ACID compliance critical
Product Catalog	MongoDB	Flexible attributes
IoT Sensor Data	TimescaleDB	Time-series optimized
Search	Elasticsearch	Full-text search needed
Session/Cache	Redis	Low-latency required
Supply Chain Relationships	Neo4j	Complex relationship queries

Synchronization Approach:

Use event-driven synchronization with Kafka. When order data changes in PostgreSQL,

publish OrderUpdated events. When product catalog changes in MongoDB, publish ProductUpdated events. Consumers update Elasticsearch, invalidate Redis caches, and maintain read models.

Consistency Trade-offs:

- Inventory counts: Strong consistency during checkout (read from PostgreSQL)
 - Search: Eventual consistency (acceptable seconds of lag)
 - Analytics: Batch consistency (daily ETL to warehouse)
-

Section 3.2: Designing for AI Integration

The Role of AI in Modern Systems

Artificial Intelligence has moved from experimental technology to essential capability. Modern systems are expected to predict, recommend, optimize, and automate—not just record and retrieve. For the Smart Inventory Platform, AI enables capabilities that would be impossible with deterministic rules alone:

- **Demand Forecasting:** Predict future inventory needs based on historical sales, seasonality, promotions, and external factors
- **Anomaly Detection:** Identify unusual patterns in inventory movement that might indicate theft, damage, or process breakdown
- **Damage Detection:** Analyze images of returned goods to classify damage severity and determine disposition
- **Fulfillment Optimization:** Dynamically route orders to optimal fulfillment locations based on inventory, labor, and transportation costs
- **Recommendation:** Suggest additional products to customers based on purchase history and current inventory

But integrating AI into operational systems requires careful architectural design. AI components have different characteristics than traditional transactional components: they may be slower, less deterministic, require specialized infrastructure, and need continuous retraining.

Three AI Integration Patterns

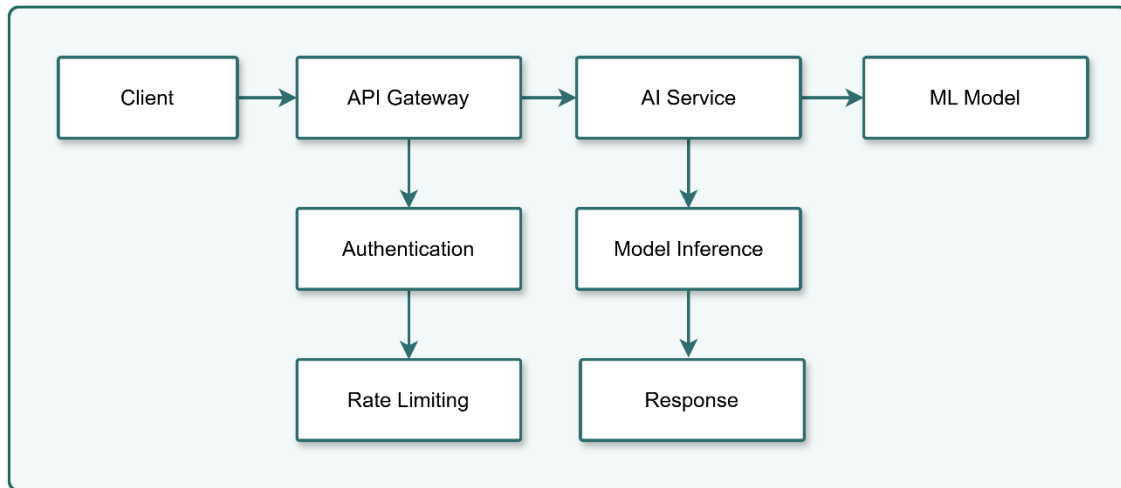
We'll explore three fundamental patterns for integrating AI into your architecture, each suited to different use cases.

Pattern 1: Request-Reply AI

Description:

In the request-reply pattern, a client sends a synchronous request to an AI service and waits for a prediction response. This is the simplest integration pattern, analogous to calling any other API.

Architecture:



Flow:

1. Client sends request with input data (e.g., product ID and date for demand forecast)
2. API Gateway authenticates request, checks rate limits
3. AI Service preprocesses input, calls ML model
4. Model performs inference and returns prediction
5. AI Service postprocesses, adds metadata (confidence scores, model version)
6. Response returned synchronously to client

Optimal Use Cases:

- Real-time predictions needed immediately
- Low to moderate latency requirements (100-500ms acceptable)
- Predictions based on current context
- Examples: Demand forecast at order placement, fraud detection during checkout, real-time recommendations

Implementation Considerations:

Model Hosting Options:

- **Managed ML Platforms:** AWS SageMaker, Azure ML, Google Vertex AI handle infrastructure, scaling, monitoring
- **Containerized Models:** Package model in Docker, deploy to Kubernetes, manage yourself
- **Serverless Functions:** AWS Lambda with container support for lightweight models

Caching:

Cache predictions for identical requests to reduce inference costs and latency. Use Redis with TTL based on prediction volatility.

Fallback Strategies:

When AI service unavailable, fallback to simpler logic—use historical averages for demand, default rules for recommendations. Circuit breakers prevent cascading failures.

Monitoring:

Track prediction latency, error rates, and data drift. Model performance degrades over time as real-world patterns change.

Code Example (Python/Flask):

```
python
@app.route('/predict-demand', methods=['POST'])
def predict_demand():
    try:
        data = request.get_json()
        product_id = data['product_id']
        forecast_date = data['date']

        # Check cache
        cache_key = f"forecast:{product_id}:{forecast_date}"
        cached = redis.get(cache_key)
        if cached:
            return jsonify(json.loads(cached))

        # Prepare features for model
        features = feature_engineering(product_id, forecast_date)
```

```

# Call SageMaker endpoint
response = sagemaker_runtime.invoke_endpoint(
    EndpointName='demand-forecast-model',
    Body=json.dumps(features),
    ContentType='application/json'
)
prediction = json.loads(response['Body'].read())

# Cache result (TTL 1 hour for daily forecasts)
result = {
    'product_id': product_id,
    'date': forecast_date,
    'predicted_demand': prediction['value'],
    'confidence_interval': prediction['interval'],
    'model_version': prediction['model_version']
}
redis.setex(cache_key, 3600, json.dumps(result))

return jsonify(result)

except Exception as e:
    # Fallback to historical average
    fallback = get_historical_average(product_id, forecast_date)
    return jsonify({
        'product_id': product_id,
        'date': forecast_date,
        'predicted_demand': fallback,
        'fallback': True
    })

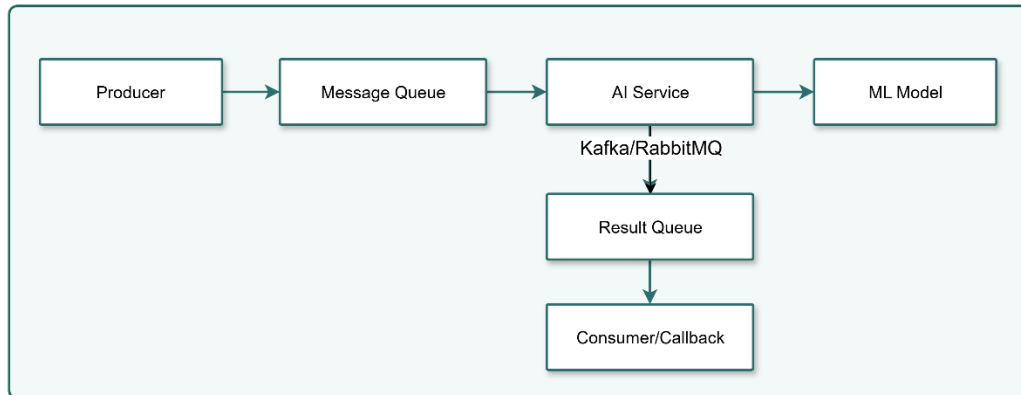
```

Pattern 2: Event-Driven AI

Description:

In the event-driven pattern, requests are sent asynchronously via message queues. The client doesn't wait for an immediate response; instead, results are delivered later through a callback, webhook, or by polling.

Architecture:



Flow:

1. Producer publishes event with input data (e.g., image for damage detection)
2. Message queue persists event, ensures delivery
3. AI Service consumes event, processes asynchronously
4. Model performs inference (may be long-running)
5. Result published to result queue or callback endpoint
6. Original producer (or another service) receives result

Optimal Use Cases:

- Long-running predictions (> 1 second)
- Batch processing of multiple items
- When client doesn't need immediate response
- When processing can be decoupled from user interaction
- Examples: Image analysis for damage detection, bulk demand forecasting for thousands of products, document processing

Implementation Considerations:

Message Durability:

Configure queues for persistence—messages should survive broker restarts. Use acknowledgments to ensure processing completion.

Dead Letter Queues:

Messages that fail processing repeatedly should be moved to dead letter queues for manual inspection.

Idempotent Processing:

Same message might be delivered twice. Ensure processing is idempotent—check if already processed before acting.

Result Notification:

- **Push to Callback:** Producer provides callback URL when submitting
- **Store for Polling:** Producer checks result store periodically
- **Event Publication:** Result published as new event for any interested consumer

Code Example (Python with Kafka):

python

Producer: Submit image for damage detection

```
def submit_damage_detection(image_id, image_bytes, callback_url):
```

```
    event = {
        'image_id': image_id,
        'image_bytes': base64.b64encode(image_bytes).decode(),
        'callback_url': callback_url,
        'submitted_at': datetime.now().isoformat()
    }
```

```
    producer.send('damage-detection-requests', event)
    producer.flush()
```

```
    return {'status': 'submitted', 'image_id': image_id}
```

Consumer: Process damage detection

```
def process_damage_detection():
```

```
    consumer.subscribe(['damage-detection-requests'])
```

```
    for message in consumer:
        event = message.value
        image_id = event['image_id']
```

```

try:
    # Check if already processed (idempotency)
    if redis.sismember('processed_images', image_id):
        consumer.commit()
        continue

    # Decode image
    image_bytes = base64.b64decode(event['image_bytes'])

    # Call ML model (could be local or SageMaker)
    result = damage_detection_model.predict(image_bytes)

    # Prepare response
    response = {
        'image_id': image_id,
        'damage_detected': result['damage'],
        'damage_type': result.get('damage_type'),
        'confidence': result['confidence'],
        'processed_at': datetime.now().isoformat()
    }

    # Send callback
    if event.get('callback_url'):
        requests.post(event['callback_url'], json=response)

    # Mark processed
    redis.sadd('processed_images', image_id)

    # Commit offset
    consumer.commit()

except Exception as e:
    # Log error, message will be retried
    logger.error(f"Failed to process {image_id}: {e}")
    # After retries exhausted, move to DLQ

```

Pattern 3: AI User Interface Design

Description:

This pattern focuses on how AI predictions are presented to users and how user feedback is collected to improve models. The UI is not just a display mechanism—it's a critical component of the human-AI collaboration.

Key Principles:

Show Confidence:

Users need to know how certain the model is. A prediction with 95% confidence should be presented differently than one with 60% confidence. Visual cues (color, size, explicit percentage) help users calibrate trust.

Provide Alternatives:

If the model predicts X, consider showing Y and Z as alternatives with their confidence scores. Users may have context the model lacks and can choose appropriately.

Enable Feedback:

Allow users to correct predictions, flag errors, or provide additional context. This feedback is gold for retraining and improving models.

Explain Decisions:

Where possible, provide explanations for predictions. "This item was classified as damaged because of scratches detected in the lower-left quadrant" helps users understand and trust (or question) the system.

Design for Collaboration:

The goal isn't to replace human judgment but to augment it. Design interfaces where AI handles routine cases and humans focus on exceptions, with clear handoffs between automated and manual processing.

UI Components for AI Features:

Confidence Indicators:

- Progress bars showing confidence percentage
- Color coding (green=high, yellow=medium, red=low)
- Explicit percentages with thresholds

Alternative Displays:

- Dropdown or carousel of top predictions
- Side-by-side comparison of alternatives
- Explanations of why alternatives were considered

Feedback Mechanisms:

- Thumbs up/down on predictions
- Correction fields ("Actual demand was...")
- Report issue button with free text
- Flag for model retraining

Example: Demand Forecast UI

Demand Forecast: Product XYZ-123

Predicted Demand: 245 units
Confidence: 92%
Range: 220 - 270 units

Alternative Scenarios:

Conservative
210 units
80% conf

Moderate
245 units
92% conf

Optimistic
290 units
75% conf

Factors Considered:

- ✓ Seasonality: +15% (holiday season)
- ✓ Promotion: Next week's sale
- ✓ Trend: +5% year-over-year
- ✗ Out of stock at competitor (not in model)

Was this forecast accurate?

☐ Yes ☐ No

Example: Damage Detection UI

Damage Detection Results

Image Preview

[IMG]

Damage Detected: YES
Confidence: 87%

Damage Type:

- Screen crack (92%)
- Corner dent (76%)
- Scratches (45%)

Affected Area:
[Image with bounding box]

Suggested Disposition:

- Warranty claim
- Repair - Screen
- [Alternative]

Correct? ☒ ☐

If incorrect, please indicate actual condition:
☐ No damage ☐ Different type ☐ Other

Choosing the Right AI Pattern

Pattern	Latency	Use Case	Complexity	Feedback Loop
Request-Reply	Low (ms)	Real-time decisions	Low	Immediate (user action)
Event-Driven	High (sec-min)	Batch processing	Medium	Delayed (async)
UI-Focused	N/A	Human-in-loop	Medium	Direct user input

For the Smart Inventory Platform, you'll likely use all three:

- **Request-Reply:** Demand forecast during order placement
- **Event-Driven:** Batch damage detection on returned goods
- **UI-Focused:** Presenting forecasts to inventory planners for review

Section 3.3: Integration Architecture

The Integration Challenge

No system exists in isolation. The Smart Inventory Platform must integrate with:

- **HRIS** for employee status changes (onboarding, offboarding)
- **Procurement Systems** for purchase orders and vendor data
- **Finance Systems** for depreciation and budgeting
- **MDM Solutions** (JAMF, Intune) for device compliance
- **Store POS Systems** for real-time sales data
- **Warehouse Automation** for inventory movements
- **Supplier Portals** for order status and returns
- **Communication Tools** (Slack, Teams) for notifications

Integration architecture is the discipline of connecting these systems in a way that is reliable, maintainable, and scalable.

Three Pillars of Integration Architecture

1. API-Led Connectivity

API-led connectivity organizes APIs into three logical layers, each serving a distinct purpose.

System APIs:

These provide access to core systems of record. They isolate the underlying system's complexity, exposing a stable interface even as the backend changes. System APIs should be coarse-grained and business-focused, not simple database CRUD wrappers.

Example: A System API for Snipe-IT might expose `getAsset(assetId)`, `assignAsset(assetId, userId)`, `getUserAssets(userId)` rather than raw table access.

Process APIs:

These orchestrate business processes that span multiple systems. Process APIs don't contain their own data—they coordinate calls to System APIs to fulfill a business capability. They implement the "how" of a business process.

Example: A Process API for `onboardEmployee(employeeData)` might call HRIS System API to create user, Snipe-IT System API to assign equipment, and Finance System API to set up cost center allocations.

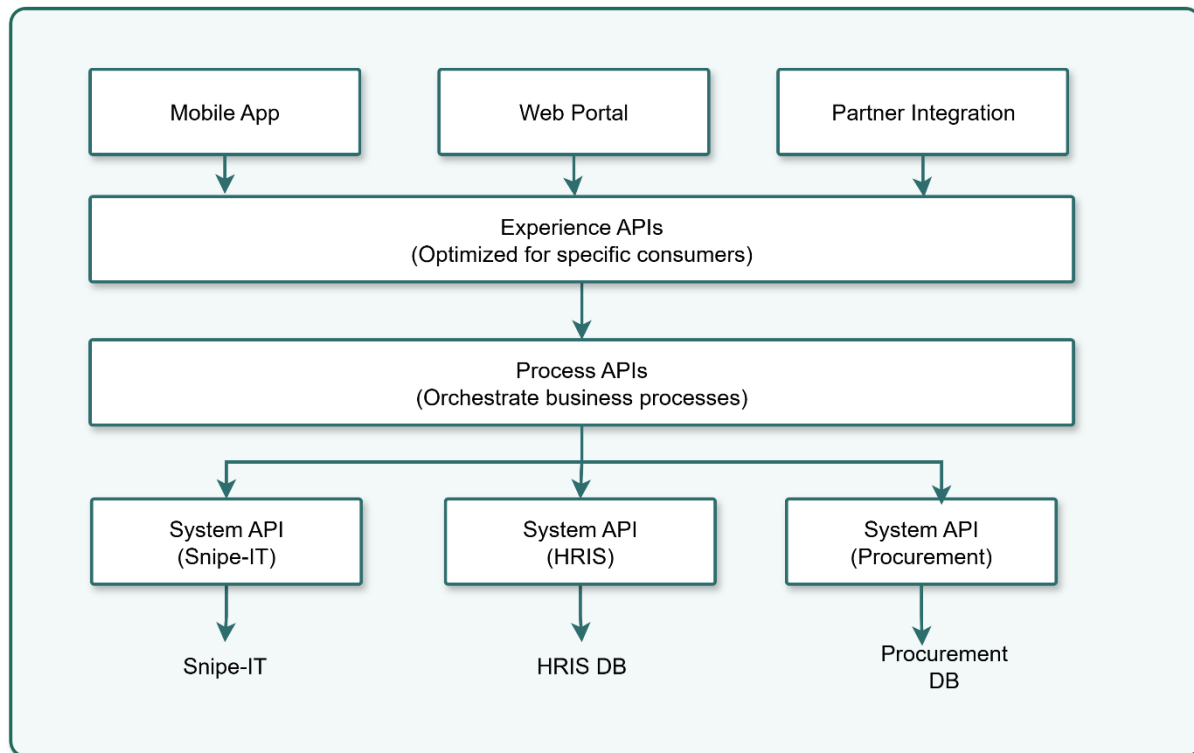
Experience APIs:

These are optimized for specific consumer experiences—mobile app, web portal, partner integration. Experience APIs reshape data from Process APIs into formats convenient for the consumer. They may combine, filter, or transform data.

Example: A Mobile Experience API might provide `getMyAssets()` that returns a simplified view of assets for the current user, hiding complexity needed for IT administration.

Benefits of API-Led Connectivity:

- **Reusability:** System APIs can be used by multiple Process APIs
- **Maintainability:** Changes to backend systems only affect System APIs
- **Security:** Each layer can enforce appropriate security controls
- **Governance:** Clear ownership boundaries between layers



2. Event-Driven Architecture (EDA)

Event-driven architecture enables real-time, loosely coupled integration. Instead of services calling each other directly, they publish events when something significant happens. Other services subscribe to relevant events and react asynchronously.

Core Concepts:

- **Event:** A significant change in state—OrderPlaced, InventoryUpdated, EmployeeTerminated
- **Publisher:** Service that emits events
- **Subscriber:** Service that consumes events and reacts
- **Event Broker:** Middleware that receives events and routes to subscribers (Kafka, RabbitMQ, AWS EventBridge)

Benefits:

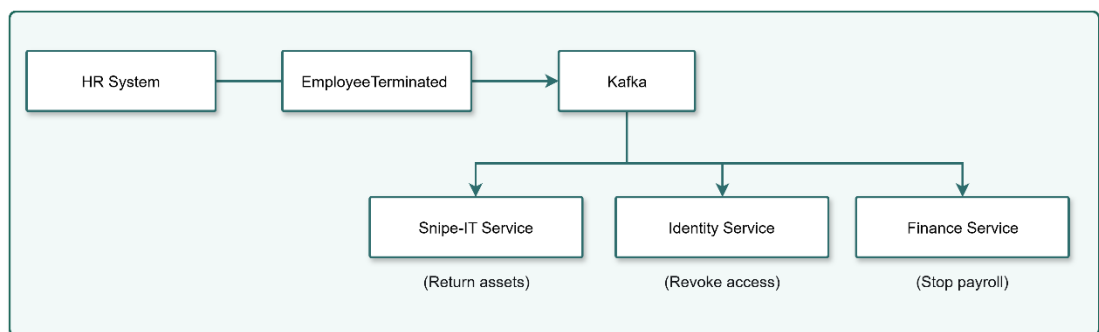
- **Loose Coupling:** Publishers don't know subscribers; subscribers don't know publishers

- **Scalability:** Multiple subscribers can process events independently
- **Real-time:** Events propagate immediately, enabling real-time reactions
- **Resilience:** If a subscriber fails, events persist in the broker for later processing

Event Types:

Event Type	Description	Example
Domain Event	Something meaningful happened in the business	OrderPlaced, InventoryDecremented
Integration Event	Data changed that other systems need to know	ProductUpdated, UserTerminated
Notification Event	Alert for human attention	LowInventoryAlert, WarrantyExpiring

Event-Driven Integration Example:



Implementation Considerations:

Event Schema:

Define a standard event format that all services can produce and consume. Include metadata (event ID, timestamp, source, version) alongside business data.

```

json
{
  "eventId": "evt_12345",
  "eventType": "EmployeeTerminated",

```

```
"eventVersion": "1.0",
"timestamp": "2024-03-15T14:30:00Z",
"source": "hr-system",
"data": {
  "employeeId": "emp_789",
  "terminationDate": "2024-03-15",
  "lastWorkingDay": "2024-03-29",
  "reason": "resignation"
}
```

Idempotency:

Same event may be delivered multiple times. Ensure subscribers can detect and handle duplicates (e.g., store processed event IDs).

Ordering:

If order matters (e.g., OrderPlaced before OrderShipped), use partitions with consistent keys to maintain ordering.

Dead Letter Queue:

Events that cannot be processed after retries should be moved to a dead letter queue for manual inspection.

3. GraphQL as a Unified API Gateway

GraphQL provides a unified interface for clients to query data from multiple underlying services and databases. Instead of multiple REST endpoints, clients have a single GraphQL endpoint and can request exactly the data they need.

Why GraphQL for Integration:**Unified Access:**

Clients don't need to know that product details come from MongoDB, inventory from PostgreSQL, and search from Elasticsearch. The GraphQL schema presents a unified view.

Reduced Over-fetching:

Clients request only the fields they need. A mobile app might request different fields than a web dashboard, all from the same endpoint.

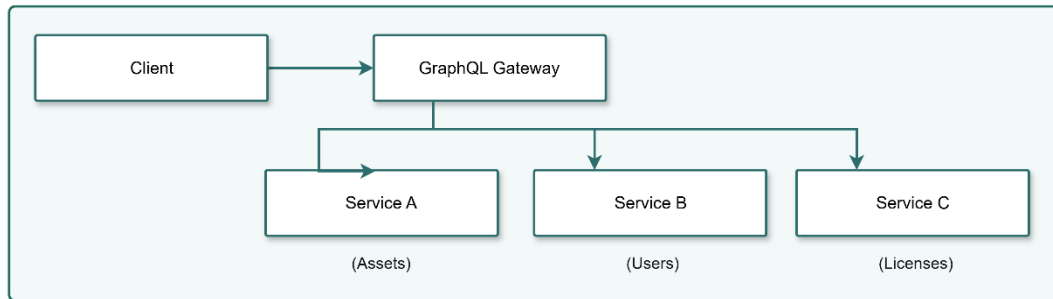
Strong Typing:

GraphQL schemas provide contract between clients and servers, with built-in validation.

Federation:

GraphQL can federate across multiple backend services, each responsible for part of the schema.

GraphQL Architecture:



GraphQL Schema Example:

graphql

```
type Asset {  
  id: ID!  
  assetTag: String!  
  name: String  
  model: Model!  
  status: Status!  
  assignedTo: User  
  location: Location  
  purchaseDate: String  
  purchaseCost: Float  
}
```

```
type User {  
  id: ID!  
  employeeId: String!  
  name: String!  
  email: String!  
  department: String  
  assets: [Asset!]!  
}
```

```
type Query {  
  asset(id: ID!): Asset  
  assetsByUser(userId: ID!): [Asset!]!
```

```

searchAssets(query: String!): [Asset!]!
me: User
}

```

Resolvers:

Each field in the schema has a resolver that knows how to fetch that data. Resolvers can call different backend services.

javascript

```

const resolvers = {
  Query: {
    asset: async (_, { id }) => {
      // Fetch from Snipe-IT System API
      return await snipeAPI.getAsset(id);
    },
    assetsByUser: async (_, { userId }) => {
      return await snipeAPI.getUserAssets(userId);
    },
    searchAssets: async (_, { query }) => {
      // Search via Elasticsearch
      return await searchService.search(query);
    }
  },
  User: {
    assets: async (user) => {
      return await snipeAPI.getUserAssets(user.id);
    }
  }
};

```

Performance Considerations:

- **N+1 Problem:** Use DataLoader to batch and cache database queries
- **Complexity Limits:** Implement query depth limiting and cost analysis
- **Caching:** Cache at HTTP level, resolver level, and database level

Integration Patterns Comparison

Pattern	Best For	Strengths	Weaknesses
API-Led Connectivity	Request-response, synchronous workflows	Clear boundaries, reusability, governance	Can become chatty, synchronous coupling
Event-Driven	Real-time updates, loose coupling	Scalable, resilient, real-time	Eventual consistency, debugging complexity
GraphQL	Client data aggregation	Flexible queries, strong typing, single endpoint	Query complexity risk, caching challenges

For the Smart Inventory Platform, you'll likely use all three:

- **API-Led** for core business capabilities (onboarding, offboarding workflows)
 - **Event-Driven** for real-time inventory updates and notifications
 - **GraphQL** for mobile app and web portal data access
-

Module Summary

Key Takeaways

Polyglot Persistence

- No single database is optimal for all use cases
- Match database technology to data requirements: relational for transactions, document for catalogs, time-series for IoT, search for lookup, graph for relationships
- Synchronization across databases introduces complexity—use Change Data Capture or event-driven approaches
- Accept eventual consistency where appropriate to simplify architecture

AI Integration

- Three patterns serve different use cases: Request-Reply for real-time, Event-Driven for async, UI-Focused for human-in-loop
- Design AI interfaces with confidence indicators, alternatives, and feedback mechanisms
- Plan for model versioning, monitoring, and retraining from the start
- Always have fallback logic when AI services are unavailable

Integration Architecture

- API-led connectivity organizes APIs into System, Process, and Experience layers for reusability and maintainability
- Event-driven architecture enables real-time, loosely coupled integration with Kafka or similar
- GraphQL provides unified access to multiple backend services with flexible client queries
- Choose integration patterns based on use case requirements, not familiarity or preference

Connection to Course Project

In your project, you should now be able to:

- Determine which data requires which database technology in your polyglot architecture
- Identify where AI can add value and select appropriate integration patterns
- Design integration between Snipe-IT and other systems using API-led and event-driven approaches
- Consider how GraphQL might simplify client access to your platform

Next Steps

Module 4 will move from design to implementation